

Exception Handling & Debugging

Prof. Ashwani Mishra

Assistant Professor
AIML Department

What Are Exceptions in Java?

- An exception is an event that disrupts the normal flow of instructions during program execution.
- Java represents exceptions as objects that belong to the Throwable class hierarchy.
- When an exceptional condition occurs, the JVM creates an exception object and searches for a handler.
- If no handler is found, the default behavior prints a stack trace and terminates the thread.
- Proper exception handling makes programs robust, readable, and easier to maintain in production.

Figure 4.1 Java Exception Class Hierarchy



Figure 4.1 Java Exception Class Hierarchy

ERROR	EXCEPTION
• <i>Errors are irreversible.</i>	• <i>Exceptions are possible to recover from exception.</i>
• <i>Errors are of unchecked type</i>	• <i>Exceptions can be either checked or unchecked.</i>
• <i>This happens at runtime.</i>	• <i>Exceptions can happen at compile and runtime both.</i>
• <i>This usually caused by the environment on which our program runs.</i>	• <i>This caused by problems in the code.</i>

Figure 4.2 Errors vs Exceptions

Checked vs Unchecked Exceptions

'Difference: Checked vs. Unchecked Exceptions in Java (for Students)'

★ 1. Definitions:

- Checked Exception: These must be 'checked' or handled by your code. Java makes you prepare for them.
- Unchecked Exception: These are 'not checked' before running. They usually mean a bug or a logic error.

★ 2. Superclass (The parent):

- Checked: Their parent is `java.lang.Exception` (but not `java.lang.RuntimeException`).
- Unchecked: Their parent is `java.lang.RuntimeException` (which itself inherits from `Exception`).

★ 3. Compile-time Rule:

- Checked: Compile-time error if not handled with try-catch or declared with throws. The code won't compile.*
- Unchecked: No compile-time check. You can handle them, but Java doesn't force you. The code will compile but might crash later.*

★ 4. Simple Real-Life Example:

- Checked (Getting your license): Before driving, you must check if you have a license. It's a rule. The system checks before you can go. (Like an `IOException`).
- Unchecked (Running): You can just start running. If you trip and fall, it's a mistake you made while running (like a `NullPointerException`). No one stopped you before you started.

★ 5. Simple Code Example (Student easy):

- Checked:

```
// Cannot compile without try-catch or throws
// import java.io.FileNotFoundException;
public void openFile() {
    // Must handle FileNotFoundException
    // try { ... }
}
```

- Unchecked: // This compiles fine, but crashes at runtime

```
public void showName(String name) {
    System.out.println(name.length()); // What if name is null? NullPointerException
}
```

★ Feature	Checked Exception	Unchecked Exception
Handled?	Must be	No (but recommended)
Parent	Exception	<code>RuntimeException</code>
Compile Time	Fails	Succeeds
Example	<code>IOException</code> , <code>SQLException</code>	<code>NullPointerException</code> , <code>ArithmeticException</code>

* Remember: Checkeds are forced, Uncheckedds are mistakes! *

Figure 4.3 Checked vs Unchecked Exceptions

Common Java Exceptions

- `NullPointerException` occurs when code dereferences a null object reference unexpectedly.
- `ArrayIndexOutOfBoundsException` is thrown when an array is accessed with an invalid index.
- `ArithmeticException` typically results from integer division by zero in Java expressions.
- `NumberFormatException` appears when `Integer.parseInt` receives a non-numeric string argument.
- `IOException` signals failure during file reading, writing, or other stream-based I/O operations.

Figure 4.4 Common Java Exceptions

Frequently Encountered Java Exceptions

Exception	Typical Cause	Prevention Tip
NullPointerException	Calling method on null reference	Check null before use
ArrayIndexOutOfBoundsException	Invalid array index	Validate index bounds
ArithmeticException	Division by zero	Test divisor before divide
NumberFormatException	Invalid string to number parse	Validate input format
IOException	File or stream I/O failure	Use try-with-resources

Figure 4.4 Common Java Exceptions

Hierarchy of Java Exceptions

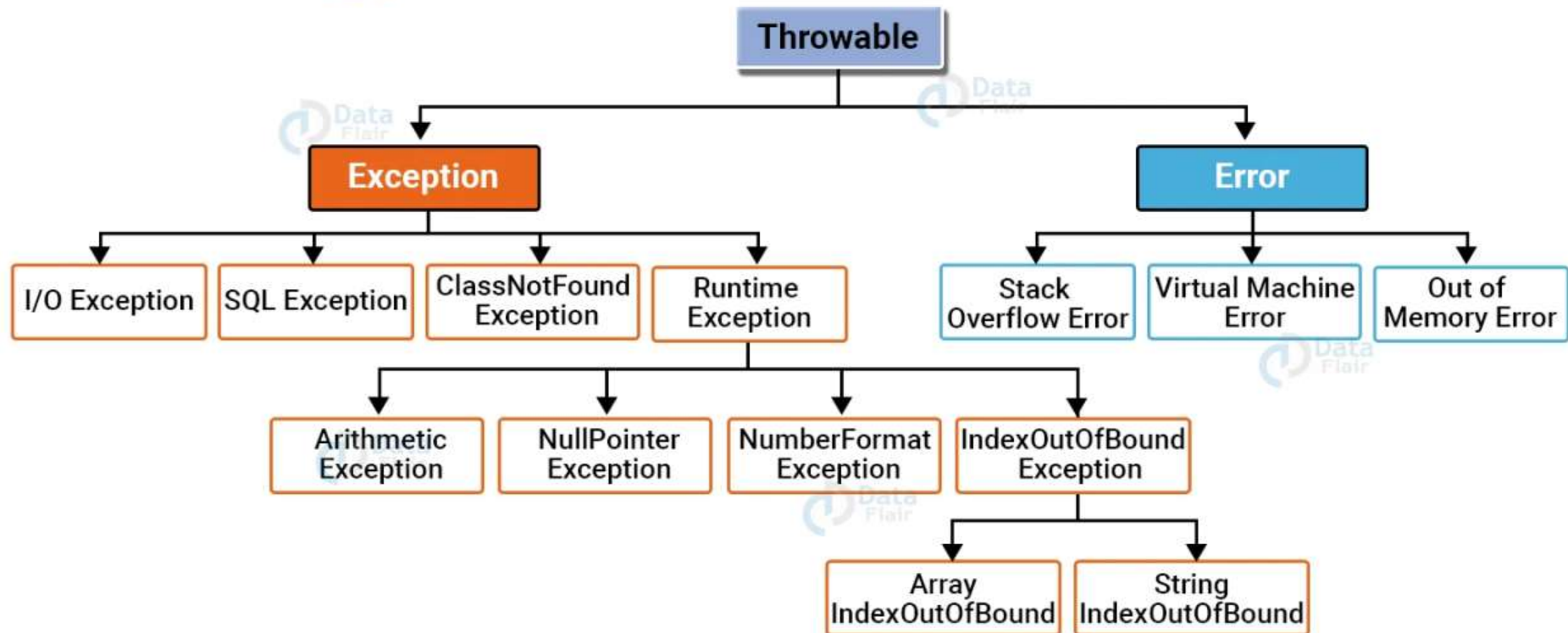


Figure 4.5 Complete Throwable Hierarchy

Structure of try-catch-finally

- The try block encloses code that might throw one or more exceptions during execution.
- Each catch block handles a specific exception type and runs only when that type is thrown.
- Multiple catch blocks can follow a single try block to handle different failure scenarios.
- The finally block always executes, whether an exception occurred or the try completed normally.
- Cleanup code such as closing files or releasing locks belongs in finally or try-with-resources.

Figure 4.6 try-catch-finally Execution Flow

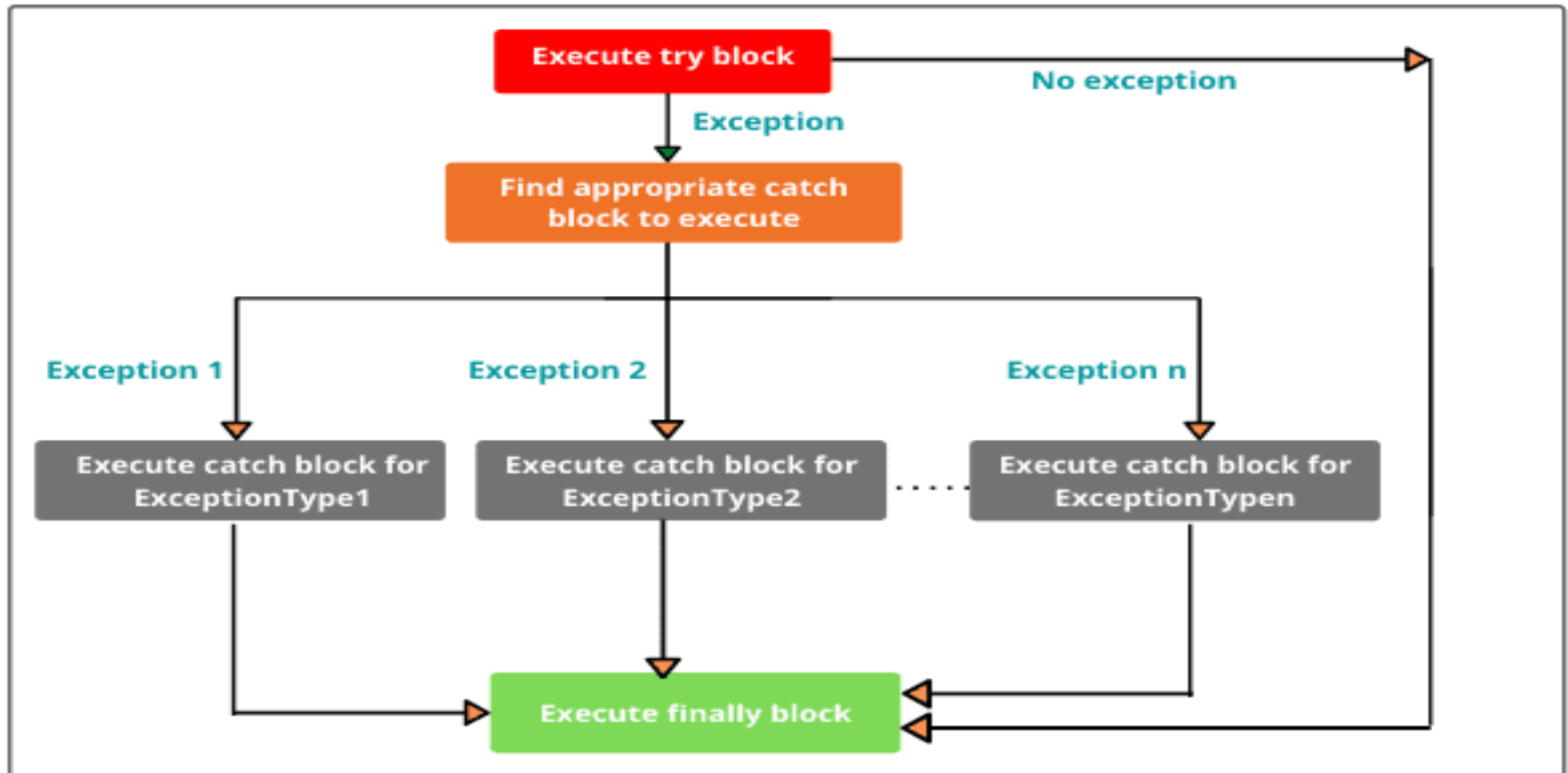


Figure 4.6 try-catch-finally Execution Flow

Basic try-catch Example

- The try block contains the risky operation that may throw an `ArithmeticException` here.
- The catch block receives the exception object and can access its message via `getMessage()`.
- Control transfers directly to catch when an exception is thrown; remaining try lines are skipped.
- After catch completes, the finally block runs to perform guaranteed cleanup actions.
- Program execution continues after the entire try-catch-finally structure without crashing.

Figure 4.7 Basic try-catch-finally Code

TryCatchExample.java

```
public class TryCatchExample {  
    public static void main(String[] args) {  
        try {  
            int result = 10 / 0;  
            System.out.println("Result: " + result);  
        } catch (ArithmeticException e) {  
            System.out.println("Cannot divide by zero: " + e.getMessage());  
        } finally {  
            System.out.println("Cleanup runs whether or not an error occurred.");  
        }  
    }  
}
```

Figure 4.7 Basic try-catch-finally Code

Multiple catch Blocks

- Separate catch blocks handle `ArrayIndexOutOfBoundsException`, `NumberFormatException`, and `ArithmeticException`.
- Catch order matters: more specific exception types must appear before their general supertypes.
- Each catch block can display a user-friendly message tailored to the particular failure type.
- This pattern is common when validating command-line arguments or parsing user input.
- The finally block confirms that validation finished regardless of which catch block executed.

Figure 4.8 Multiple catch Blocks

MultipleCatch.java

```
try {  
    String value = args[0];  
    int num = Integer.parseInt(value);  
    int result = 100 / num;  
} catch (ArrayIndexOutOfBoundsException e) {  
    System.out.println("No command-line argument provided.");  
} catch (NumberFormatException e) {  
    System.out.println("Argument is not a valid integer.");  
} catch (ArithmeticException e) {  
    System.out.println("Division by zero is not allowed.");  
} finally {  
    System.out.println("Input validation attempt finished.");  
}
```

Figure 4.8 Multiple catch Blocks

try-with-resources (Java 7+)

- Try-with-resources automatically closes resources that implement the `AutoCloseable` interface.
- Resources declared in the try header are closed in reverse order when the block exits.
- This eliminates many manual `close()` calls that were previously placed in finally blocks.
- `IOException` during file reading is still caught explicitly in the catch block below.
- Using try-with-resources reduces resource leaks and produces cleaner, safer I/O code.

try-with-resources for Auto-Closeable Resources

TryWithResources.java

```
// Auto-closes resources implementing AutoCloseable
try (FileReader reader = new FileReader("data.txt");
    BufferedReader br = new BufferedReader(reader)) {
    String line;
    while ((line = br.readLine()) != null) {
        System.out.println(line);
    }
} catch (IOException e) {
    System.out.println("File read failed: " + e.getMessage());
}
// reader and br are closed automatically, even if an exception occurs
```

Figure 4.9 try-with-resources for Auto-Closeable Resources

The throws Keyword

- Adding throws IOException to a method signature declares that the method may propagate that exception.
- Any caller of readFile must either catch IOException or declare throws in its own signature.
- The throws clause documents the checked exceptions a method does not handle internally.
- Unchecked exceptions such as NullPointerException do not require a throws declaration.
- Declaring throws pushes responsibility for recovery to the calling code or higher-level handler.

Figure 4.10 Declaring Exceptions with throws

ThrowsExample.java

```
import java.io.IOException;

public class ThrowsExample {
    // Declares that callers must handle or re-declare IOException
    public static void readFile(String path) throws IOException {
        FileReader reader = new FileReader(path);
        reader.close();
    }

    public static void main(String[] args) {
        try {
            readFile("notes.txt");
        } catch (IOException e) {
            System.out.println("Could not read file: " + e.getMessage());
        }
    }
}
```

Figure 4.10 Declaring Exceptions with throws

Custom Exception Classes

- Custom exceptions extend Exception for checked behavior or RuntimeException for unchecked behavior.
- A constructor calling super(message) stores a descriptive error message inside the exception object.
- The throw keyword creates and immediately throws a new InvalidAgeException instance at runtime.
- Business rules such as minimum age are enforced by throwing domain-specific exception types.
- Callers catch InvalidAgeException to display meaningful feedback instead of generic error text.

Figure 4.11 Custom Exception Class and throw

CustomException.java

```
// Custom checked exception
class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) {
        super(message);
    }
}

public class StudentRegistration {
    public void register(int age) throws InvalidAgeException {
        if (age < 18) {
            throw new InvalidAgeException("Student must be at least 18 years old.");
        }
        System.out.println("Registration successful for age " + age);
    }
}
```

Figure 4.11 Custom Exception Class and throw

Designing Custom Exceptions

- Choose Exception as the base class when callers must explicitly handle the failure condition.
- Choose RuntimeException when the error represents a programming bug or unrecoverable state.
- Provide meaningful constructor messages so stack traces and logs identify the root cause quickly.
- Name custom exceptions clearly, such as InvalidAgeException rather than a vague MyException class.
- Use custom exceptions to model business rules that standard Java exceptions cannot express precisely.

Reading a Stack Trace

- The first line names the exception type and includes the human-readable error message.
- Each at line shows a method call on the call stack with the source file and line number.
- Reading from top to bottom traces from the throw point outward to the entry point main.
- The deepest at line usually points to the exact statement that caused the exception.
- Stack traces are the first diagnostic tool when an uncaught exception terminates a program.

Figure 4.13 Understanding Stack Trace Output

StackTraceOutput.txt

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at TryCatchExample.main(TryCatchExample.java:5)
    at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
    at java.base/jdk.internal.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:43)

// Read bottom-to-top: main() at line 5 caused the divide-by-zero.
// File name and line number pinpoint the exact source location.
```

Figure 4.13 Understanding Stack Trace Output

IDE Debugging Tools

Common IDE Debugging Tools

Tool	Purpose	Typical Shortcut
Breakpoint	Pause at a specific line	Click gutter / F9
Step Over	Execute current line, stay in method	F8
Step Into	Enter method call on current line	F7
Watch / Variables	Monitor expression values	Debug panel
Evaluate Expression	Test code at breakpoint	Alt+F8 (IntelliJ)

Figure 4.14 IDE Debugging Tools Reference

Step-by-Step Debugging Workflow

- Set a breakpoint on the line where you suspect incorrect behavior or an exception originates.
- Launch the program in debug mode so execution pauses when the breakpoint line is reached.
- Inspect local variables and object fields in the Variables panel to verify expected values.
- Use Step Over to execute one line at a time without entering method calls on that line.
- Apply Step Into to trace inside method calls and locate the exact source of logical errors.

Figure 4.15 IDE Debugging Workflow

IDE Debugging Workflow



Figure 4.15 IDE Debugging Workflow

Effective Debugging Practices

- Reproduce the bug consistently before debugging so you can verify that a fix actually works.
- Use conditional breakpoints to pause only when a variable reaches a specific problematic value.
- Combine logging with debugging: log messages help trace behavior in environments without an IDE.
- Fix the root cause shown by the stack trace rather than masking symptoms with empty catch blocks.
- Retest edge cases after fixing to ensure the exception handler covers all related failure paths.

Parul[®]
University

NAAC
GRADE **A++**



<https://paruluniversity.ac.in/>

